

Producer-Consumer Synchronization Service

Final Project Report

ICS433 Operating Systems

Group 2 | Section F61

Team Member	Student ID
Reema Ibrahim AlQahtani	202244660
Sara Azmi	202271160
Sarah Imad	202243500
Hawra Ali	202333090

Submission Date: May 4, 2026

Table of Contents

1.	Introduction	3
2.	Problem Statement and Motivation	3
3.	System Design	4
4.	Implementation Details	6
5.	Algorithms Used	8
6.	Tools and Technologies	9
7.	Results and Testing	9
8.	Challenges and Solutions	12
9.	Future Improvements	13
10.	Team Contribution	13

1. Introduction

This report documents the complete design, implementation, testing, and validation of a **Producer-Consumer Synchronization Service** developed for ICS433, Operating Systems. The system implements a classic concurrent systems problem: multiple producer processes and multiple consumer processes sharing a bounded buffer, coordinated exclusively through POSIX inter-process communication (IPC) primitives on Linux.

The solution is implemented in C (C11) and uses POSIX shared memory and named semaphores as the sole communication and synchronization mechanisms. All producers and consumers are independent operating system processes, not threads, reflecting a realistic multi-process operating system design.

2. Problem Statement and Motivation

The producer–consumer problem is one of the fundamental synchronization problems in operating systems. In its general form, one or more *producers* generate data items and place them into a shared bounded buffer, while one or more *consumers* remove and process those items. The core challenge is coordinating concurrent access so that:

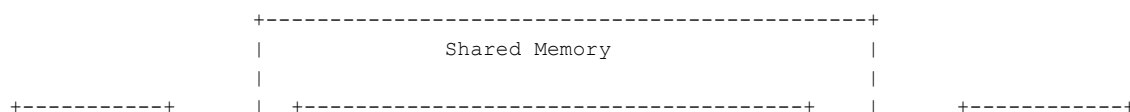
- Producers do not insert into a full buffer (overflow).
- Consumers do not remove from an empty buffer (underflow).
- No item is lost (produced but never consumed).
- No item is consumed more than once (duplicate consumption).
- No race conditions occur when multiple processes access shared state simultaneously.
- The system starts cleanly, terminates cleanly, and leaves no orphaned IPC resources.

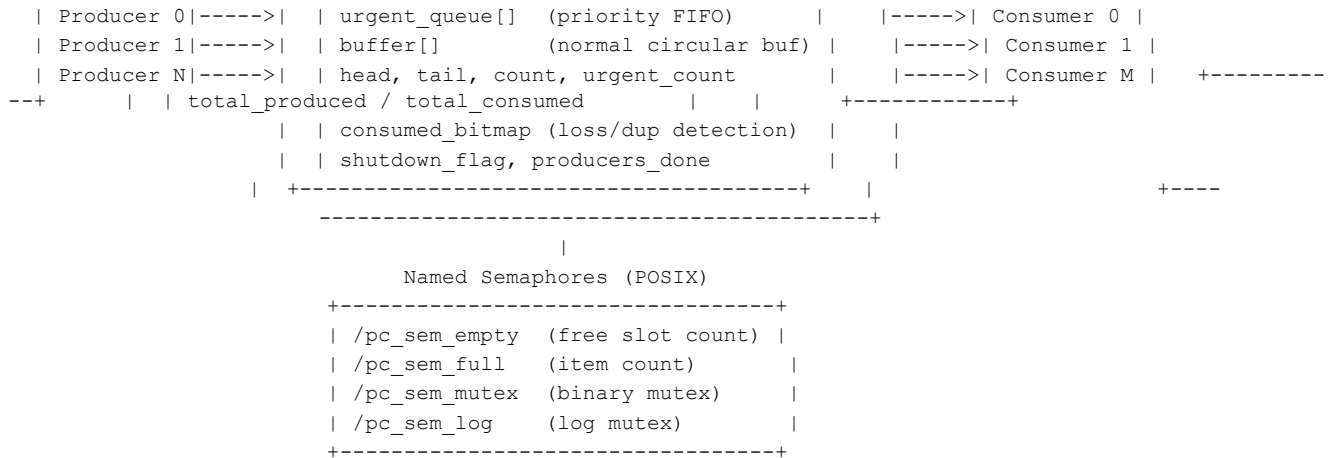
The motivation for this implementation is to demonstrate mastery of POSIX IPC primitives (shared memory and semaphores), process management (fork/exec), concurrent process synchronization, and robust system design with validation.

3. System Design

3.1 Architecture Overview

The system consists of four components compiled into three separate binaries. A **controller** process serves as the entry point: it creates shared memory and semaphores, then uses fork+exec to spawn N producer processes and M consumer processes. After all children terminate it runs a validation pass and cleans up all IPC resources.





Controller: spawns all children, waits, validates, cleans up.

Figure 1: System Architecture Diagram

3.2 Shared Memory Layout

A single **SharedBuffer** struct is mapped into POSIX shared memory via `shm_open` and `mmap`. It contains all state shared between processes:

- **buffer[MAX_BUFFER_SIZE]** the normal circular queue (FIFO) for `PRIORITY_NORMAL` items.
- **urgent_queue[MAX_BUFFER_SIZE]** a separate bounded FIFO for `PRIORITY_URGENT` items.
- **head, tail, count** normal queue indices and occupancy count.
- **urgent_head, urgent_tail, urgent_count** urgent queue indices and occupancy.
- **total_produced, total_consumed, items_requested** accounting counters (all under mutex).
- **consumed_bitmap** one bit per sequence number for loss and duplicate detection.
- **shutdown_flag** `STATUS_RUNNING` or `STATUS_SHUTDOWN`.
- **producers_done, num_producers** for last-producer-triggers-shutdown logic.

3.3 Semaphore Design

Three semaphores implement the classic bounded-buffer synchronization protocol:

Semaphore	Initial Value	Purpose
/pc_sem_empty	buffer_size (N)	Counts free buffer slots. Producers wait here when buffer is full.
/pc_sem_full	0	Counts filled slots. Consumers wait here when buffer is empty.
/pc_sem_mutex	1	Binary mutex protecting all shared state modifications.
/pc_sem_log	1	Cross-process mutex preventing interleaved log output.

Table 1: POSIX Named Semaphores

3.4 Process Model

The controller forks and execs each producer and consumer as a separate process. Consumers are spawned first (they block on `sem_full` immediately). After a 50ms pause for consumers to initialize, producers are spawned. The controller then waits for all producers to exit, posts wake tokens for consumers, waits for all consumers, and finally runs validation and cleanup.

3.5 Item Distribution

The controller distributes `total_items` as evenly as possible across `N` producers. If `total_items` is not divisible by `N`, the first (`total_items mod N`) producers each receive one extra item. This ensures every item is produced exactly once with no duplicates in the distribution.

4. Implementation Details

4.1 Build System

The Makefile compiles three binaries (controller, producer, consumer) with a shared logger object file. Compiler flags enforce strict C11 compliance:

```
gcc -Wall -Wextra -Wpedantic -O2 -g -std=c11 -D_GNU_SOURCE -I./include
Linked: -lrt -lpthread
```

The build completes with **zero warnings and zero errors**.

4.2 Producer Process (producer.c)

Each producer is launched with three arguments: `producer_id`, `num_items`, `urgent_pct`. It attaches to shared memory and opens the semaphores, then executes the following sequence for each item:

1. Check `shutdown_flag` stop early if shutdown was requested externally.
2. Call `simulate_produce()` sleep 0–7 ms to simulate real work and create interleaving.
3. `sem_wait(empty)` block if all buffer slots are occupied.
4. `sem_wait(mutex)` enter critical section.
5. Claim next unique sequence number (`shm->total_produced + 1`).
6. Build Item struct: `seq_num`, `value (= seq_num)`, `producer_id`, `priority`, `timestamp`.
7. Insert into `urgent_queue` if `PRIORITY_URGENT`, else into circular buffer.
8. Increment `total_produced`.
9. `sem_post(mutex)` exit critical section.
10. `sem_post(full)` signal that one item is now available.
11. Log the produced item with its sequence number, priority, and slot.

When all items are produced, the producer increments `producers_done` under the mutex. If it is the last producer, it also sets `shutdown_flag = STATUS_SHUTDOWN`.

4.3 Consumer Process (consumer.c)

Each consumer is launched with one argument: `consumer_id`. It attaches to shared memory, opens semaphores, and loops executing:

1. `sem_wait(full)` block if no items are available.
2. `sem_wait(mutex)` enter critical section.
3. Check exit condition: if both queues are empty and `shutdown_flag` is set, exit.
4. Remove item drain `urgent_queue` completely before touching normal buffer.
5. Check `consumed_bitmap` for this `seq_num`: log a warning if already set (duplicate).
6. Set the bit in `consumed_bitmap` and increment `total_consumed`.
7. `sem_post(mutex)` exit critical section.
8. `sem_post(empty)` signal that one slot is now free.
9. Log the consumed item with latency (time from production to consumption).
10. Call `simulate_consume()` sleep 0–7 ms to simulate real work.

4.4 Process-Safe Logging (logger.c)

All processes log to both `stdout` and `logs/pc_service.log`. A dedicated semaphore (`SEM_LOG`) acts as a cross-process mutex so entries from concurrent processes never interleave. Each log line includes millisecond timestamp, severity level, and process ID.

```
[2026-05-04 13:31:09.532] [INFO ] [622] Consumer 0 | seq=1 | val=1 | priority=NORMAL | slot=0 | producer=0 |
```

4.5 IPC Lifecycle

On startup, the controller always unlinks any stale IPC objects from a previous run before creating new ones (making the system self-healing). On exit whether normal or signal-triggered the controller calls `munmap`, `shm_unlink`, and four `sem_unlink` calls. Child processes only `sem_close` and `munmap`; they never unlink.

5. Algorithms Used

5.1 Classic Three-Semaphore Bounded Buffer

The core synchronization protocol uses three semaphores `empty`, `full`, and `mutex` in the classic Dijkstra pattern. Producers acquire `empty` then `mutex`; consumers acquire `full` then `mutex`. This ordering is consistent across all code paths, preventing deadlock while providing mutual exclusion and blocking on capacity.

5.2 Two-Tier Priority Queue

Priority is implemented as two parallel queues inside shared memory rather than a single sorted structure. This provides $O(1)$ insert and $O(1)$ remove at either priority level. Consumers check `urgent_count` first and drain the urgent queue completely before pulling from the normal buffer. The `sem_empty` semaphore counts total capacity across both queues since every item regardless of priority consumes one logical slot.

5.3 Circular Buffer (FIFO)

Both queues use a circular array with head and tail indices advancing modulo `buffer_size`. This provides $O(1)$ enqueue and $O(1)$ dequeue with no memory allocation or copying. The index arithmetic is encapsulated in the `CIRC_NEXT` macro:

```
#define CIRC_NEXT(idx, size) (((idx) + 1) % (size))
```

5.4 Bitmap-Based Validation

The `consumed_bitmap` is a `uint8_t` array of size $(\text{MAX_ITEMS}/8)+1$. Each bit corresponds to one sequence number (1-based). `BITMAP_SET` uses bitwise OR to mark a sequence number; `BITMAP_TEST` uses bitwise AND to check it. A full scan of bits 1..`total_produced` after all processes finish provides mathematical certainty of zero data loss.

```
#define BITMAP_SET(bm, seq) ((bm)[((seq)-1)/8] |= (1u << (((seq)-1)%8)))
#define BITMAP_TEST(bm, seq) ((bm)[((seq)-1)/8] & (1u << (((seq)-1)%8)))
```

5.5 Cascade Shutdown Token Propagation

A subtle shutdown challenge: consumers blocked on `sem_wait(full)` must be unblocked after producers finish. The solution uses cascade propagation: the controller posts one token per consumer immediately after all producers exit. Each consumer that decides to exit then posts one additional token to unblock a sibling, propagating the wake-up chain to all consumers without busy-waiting.

6. Tools and Technologies

Technology	Role
C (C11)	Core implementation language
GCC 13	Compiler with -Wall -Wextra -Wpedantic -O2 -g
GNU Make	Build system (Makefile with all/clean/run/runbig/runurgent targets)
POSIX shm_open/mmap	Shared memory IPC between processes
POSIX sem_open	Named semaphores for synchronization and logging
fork / exec	Process creation for producers and consumers
clock_gettime	High-resolution timestamps for logging and latency measurement
sigaction	Signal handling for graceful SIGINT/SIGTERM shutdown
waitpid	Reaping child processes; handles EINTR restarts
Linux (Ubuntu 24)	Target OS POSIX realtime extensions required

Table 2: Technologies Used


```
[2026-05-04 13:31:09.470] [INFO ] [621]    buffer_size    = 10
[2026-05-04 13:31:09.470] [INFO ] [621]    num_producers   = 3
[2026-05-04 13:31:09.470] [INFO ] [621]    num_consumers   = 2
[2026-05-04 13:31:09.470] [INFO ] [621]    total_items     = 50
[2026-05-04 13:31:09.470] [INFO ] [621]    urgent_pct      = 20%
[2026-05-04 13:31:09.477] [INFO ] [621] Controller: spawned consumer 0 (pid=622)
[2026-05-04 13:31:09.477] [INFO ] [621] Controller: spawned consumer 1 (pid=623)
[2026-05-04 13:31:09.528] [INFO ] [621] Controller: spawned producer 0 (pid=624, items=17) [2026-05-04
13:31:09.528] [INFO ] [621] Controller: spawned producer 1 (pid=625, items=17)
[2026-05-04 13:31:09.528] [INFO ] [621] Controller: spawned producer 2 (pid=626, items=16) [2026-05-04
13:31:09.532] [INFO ] [622] Consumer 0 | seq=1 | priority=NORMAL | slot=0 | producer=0 | latency
[2026-05-04 13:31:09.532] [INFO ] [624] Producer 0 | seq=1 | priority=NORMAL | slot=0 | produced [2026-05-04
13:31:09.534] [INFO ] [623] Consumer 1 | seq=2 | priority=NORMAL | slot=1 | producer=2 | latency
[2026-05-04 13:31:09.543] [INFO ] [625] Producer 1 | seq=9 | priority=URGENT | slot=0 | produced
[2026-05-04 13:31:09.544] [INFO ] [622] Consumer 0 | seq=9 | priority=URGENT | slot=0 | latency=317us | cons
[2026-05-04 13:31:09.544] [INFO ] [622] Consumer 0 | seq=7 | priority=NORMAL | slot=6 | latency=137lus | con
... (all 50 items produced and consumed)
```

7.3 Validation Report Output (Test 1)

[illegible]

Figure 2: Validation Report Test 1 (50 items, 3 producers, 2 consumers)

7.4 Execution Summary (Test 1)

[illegible]

Figure 3: Execution Summary Test 1

7.5 Priority Queue Behaviour

The following excerpt demonstrates correct priority ordering. URGENT items (seq=9, 10) are consumed with microsecond latencies, while NORMAL items (seq=7, 8) that arrived before them waited while the urgent items drained first:

```
[13:31:09.542] Producer 0 | seq=7 | priority=NORMAL | slot=6 | produced
[13:31:09.543] Producer 1 | seq=9 | priority=URGENT | slot=0 | produced <-- urgent
[13:31:09.544] Consumer 0 | seq=9 | priority=URGENT | latency=317us | consumed <-- served first
[13:31:09.544] Consumer 0 | seq=7 | priority=NORMAL | latency=1371us | consumed <-- waited
```

Figure 4: Priority Queue Ordering URGENT items consumed before earlier NORMAL items

Dual-Queue Priority Without Sorted Insertion

Problem: A single priority-sorted buffer would require $O(n)$ insertion. Maintaining ordering across concurrent insertions from multiple producers is complex.

Solution: Two separate FIFO queues: one for NORMAL and one for URGENT items. Both provide $O(1)$ insert and remove. Consumers simply drain the urgent queue first. The `sem_empty` semaphore accounts for both queues combined.

Cross-Process Log Interleaving

Problem: Multiple concurrent processes writing to the same file and stdout produce interleaved partial lines, making logs unreadable.

Solution: A dedicated POSIX named semaphore (`/pc_sem_log`) acts as a cross-process mutex around every log write. Each entry is fully formatted into a local buffer before the lock is acquired, minimizing hold time.

Uneven Item Distribution

Problem: When `total_items` is not divisible by `num_producers`, a naive floor division silently drops the remainder, producing fewer items than requested.

Solution: The first $(total_items \bmod N)$ producers each receive one extra item ($base + 1$), and the rest receive $base$ items, distributing all items without duplication across any number of producers.

9. Future Improvements

- **Condition variables in shared memory:** Replace the three-semaphore protocol with a `pthread_mutex_t` and `pthread_cond_t` stored in shared memory (using `PTHREAD_PROCESS_SHARED`), providing a monitor-style design closer to high-level language concurrent primitives.
- **Multiple priority levels:** Extend from two tiers (NORMAL/URGENT) to an arbitrary k -level priority scheme using k separate queues, enabling finer-grained scheduling.
- **Performance benchmarking:** Measure throughput (items/second) and latency distributions across configurations, and add a `--benchmark` flag that suppresses logging overhead for accurate measurements.
- **Dynamic scaling:** Allow producers and consumers to be added or removed while the system is running by supporting runtime reconfiguration of `num_producers` and `num_consumers`.
- **Persistent logging:** Implement a rotating log file with configurable maximum size, preventing unbounded disk usage in long-running deployments.
- **GitHub Actions CI:** Add a workflow that automatically builds and runs the full test suite on every pull request, ensuring regressions are caught immediately.

10. Team Contribution

The project was divided into four parallel workstreams, with each member owning one core component end-to-end. All members participated in integration testing and code review.

Member	Area	Contributions
Reema Ibrahim AlQahtani 202244660	Controller & Project Setup	Designed and implemented <code>controller.c</code> the system entry point responsible for argument parsing, shared memory creation and initialisation (<code>shm_open</code> , <code>ftruncate</code> , <code>mmap</code>), semaphore creation, process spawning (<code>fork+exec</code>), child reaping (<code>waitpid</code>), signal handling (<code>SIGINT/SIGTERM</code>), post-run validation, and full IPC cleanup. Also authored the README, Makefile, .gitignore, and all project documentation.
Sara Azmi 202271160	Consumer Process	Designed and implemented <code>consumer.c</code> the consumption loop including <code>sem_wait/sem_post</code> sequencing, priority-aware item removal (urgent queue drains before normal buffer), bitmap marking for duplicate detection, latency calculation, graceful shutdown detection, and cascade token posting on exit. Also implemented <code>logger.c</code> and <code>logger.h</code> : the cross-process safe timestamped logging subsystem using a dedicated POSIX semaphore.
Sarah Imad 202243500	Shared Memory & Buffer Design	Designed and implemented <code>shared.h</code> the central header defining the <code>SharedBuffer</code> struct, <code>Item</code> struct, all IPC resource names, buffer/bitmap macros (<code>CIRC_NEXT</code> , <code>BITMAP_SET</code> , <code>BITMAP_TEST</code>), priority constants, and status flags. Responsible for the complete shared memory layout including the dual-queue design (normal circular buffer + urgent FIFO), accounting counters, and the <code>consumed_bitmap</code> for correctness validation.
Hawra Ali 202233090	Producer Process	Designed and implemented <code>producer.c</code> the production loop including <code>sem_wait/sem_post</code> sequencing, unique sequence number assignment under mutex, priority-based queue selection (urgent_queue vs. normal buffer), item distribution across multiple producers, early shutdown detection, and the last-producer shutdown flag mechanism. Also designed and executed all 12 test cases and wrote the automated test runner (<code>scripts/run.sh</code>).

Table 4: Team Contribution by Component

